

university of groningen

Performance Engineering: Lecture #10

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-30

Overview



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Content



1. **Memory Architecture (2)**
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Cache



Cache miss = Word not found in cache

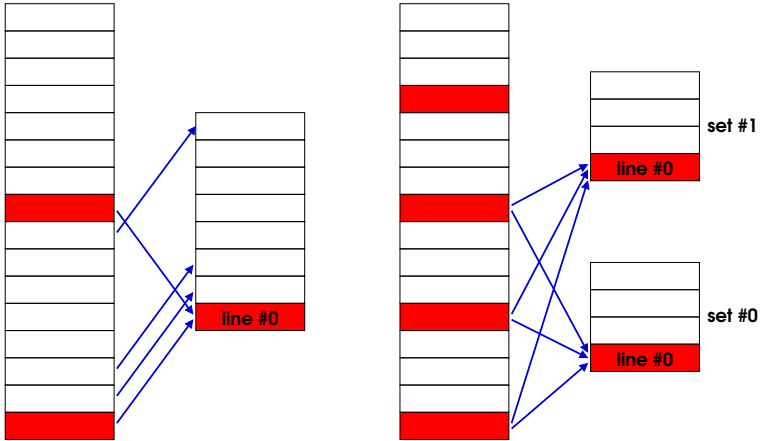
- Word has to be fetched from next memory level
- Typically a full **cache line** (=multiple words) is fetched

Cache organization: **Set associative cache**

- Match line onto a set and then place line within the set
  - Choice of set address: (address) mod (number of sets)
- Types:
  - **Direct-mapped cache**: 1 cache line per set
    - Each line has only one place it can appear in the cache
  - **Fully associative cache**: 1 set only
    - A line can be placed anywhere in the cache
  - **n-way set associative cache**: *n* cache lines per set
    - A line can be placed in a restricted set of places (1 out of *n* different places) in the cache

Cache Organization

Direct-mapped vs. 4-way set associative cache:



Cache Write Policies



Issue: How to keep cache and memory consistent?

- Cache write policy design options
  - **Write-through cache**
    - Update cache and memory
    - Memory write causes significant delay if pipeline must stall
  - **Write-back cache**
    - Update cache only and post-pone update of memory
      ☞ memory and cache will become inconsistent
    - If cache line is **dirty** then it must be committed before being replaced
    - Add dirty bit to manage this task

Issue: How to manage partial cache line updates?

- Typical solution: **write-allocate** (fetch-on-write)
  - Cache line is read before being modified

Digression: Stream Benchmark Performance



- Analyse impact of write-allocate mechanism assuming a cache line size of 64 byte and double-precision numbers
- **Scale benchmark** with arrays of length N:

$$\vec{b} \leftarrow s \cdot \vec{c}$$

  - Cache line loads: $(1+1) \cdot 8 \cdot N/64$
  - Cache line writes: $1 \cdot 8 \cdot N/64$
- **Triad benchmark**:

$$\vec{a} \leftarrow \vec{b} + s \cdot \vec{c}$$

  - Cache line loads: $(1+2) \cdot 8 \cdot N/64$
  - Cache line writes: $1 \cdot 8 \cdot N/64$
- Write-allocate may cause

$$b_{\text{triad}} \simeq \frac{9}{8} b_{\text{scale}}$$

Cache Misses



- Categories of cache misses

  Compulsory: Occurs when cache line is accessed the first time

  Capacity: Re-fetch of cache lines due to lack of cache capacity

  Conflict: Cache line was discharged due to lack of associativity

- **Average memory access time** $=$

$$\text{hit time} + \text{miss rate} \times \text{miss penalty}$$

Hit time ... Time to hit in the cache

Miss penalty ... Time to load cache line from memory

Inclusive versus Exclusive Caches

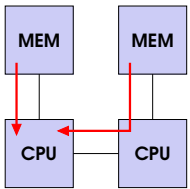


- Policies:
  - **Inclusive cache** = Data present in one cache level also present in higher cache level
    - Example: All data in L1 cache is also present in L2 cache
  - **Exclusive cache** = Data present in at most one cache level
  - Intermediate policies:
    - Data available at higher cache level is allowed to be available at lower cache levels
- Advantages of inclusive cache
  - Management of cache coherence may be simpler
- Advantages of exclusive cache
  - Duplicate copies of data in several cache levels avoided
- Example for intermediate policy:
  - L3 cache which is used as **victim cache**: cache holds data evicted from L2, but not necessarily data loaded to L2

Non-Uniform Memory Access



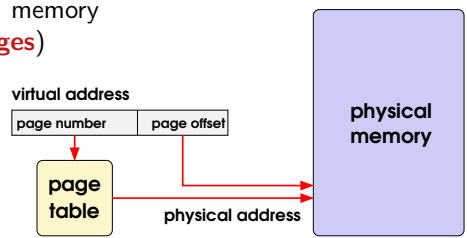
- **Non-Uniform Memory Access (NUMA)** = The data paths to the physical memory are non-uniform
- **Example**: Dual-socket node design with a shared memory space
  - A processor can load from another processor's memory



  - Memory bandwidth and access latency depend on which memory is accessed

Virtual Memory

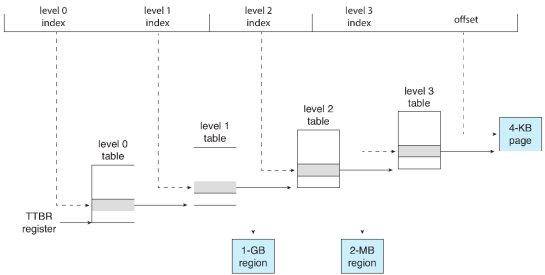
- **Virtual memory** = Memory management technique that allows each process to have its own contiguous memory address space
- Practical implementation:
  - The operating systems (supported by hardware) manages mapping between **physical memory addresses** and **virtual memory addresses**
  - To simplify this mapping the physical (virtual) memory is organised in **memory frames** (**memory pages**)
  - Memory mapping assuming equally-sized frames and pages:



Page Tables Look-up

- Hardware mechanisms that speed-up page table look-ups by
  - **Translation Lookaside Buffers (TLB)**
    - Small cache holding page table entries
  - **Hardware table walk** supporting multi-level page tables
    - Example: Arm-v8 4-Level Lookup

[Silberschatz; 2018]



Demand Paging



- **Demand paging** = Memory management technique that brings pages into memory when an executing process demands them
- Situations where demand paging will occur:
  - Virtual memory page exists but was swapped to storage
    - OS will select a memory frame, read content from storage, and create/update a page table entry
  - Virtual memory page has never been accessed
    - OS will create virtual memory page, assign a memory page, and create a page table entry
    - Note: This does not happen when virtual memory is allocated

    ☞ **First-touch policy**

Content



1. Memory Architecture (2)
2. **Memory Performance Tools**
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

Memory Tools: Introduction



- Tools to discover parameters of the memory subsystem
  - `dmidecode`
  - `hwloc-ls`
- Tools to monitor performance
  - Perf (see tutorial #1)
  - Valgrind Memcheck
  - Valgrind Cachegrind

Discovery of the Memory Subsystem: SMBIOS



- Information about the available hardware is typically stored in the **System Management BIOS (SMBIOS)**
- Linux tool for reading the SMBIOS: `dmidecode`

Example:

```
# dmidecode 3.5
Getting SMBIOS data from sysfs.
SMBIOS 3.2.0 present.
Table at 0x9109F000.

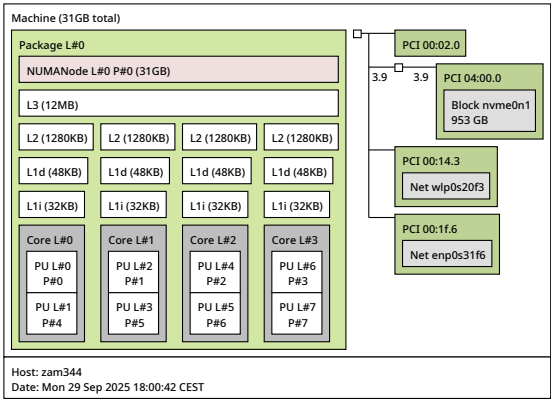
...

Handle 0x0001, DMI type 17, 92 bytes
Memory Device
        ...
        Total Width: 16 bits
        Data Width: 16 bits
        Size: 4 GB
        ...
        Bank Locator: BANK 0
        Type: LPDDR4
        ...
        Speed: 4267 MT/s
        ...
        Rank: 2
        Configured Memory Speed: 4267 MT/s
        ...
```

Discovery of the Memory Subsystem: hwloc

- **Portable Hardware Locality (hwloc)** was developed in the context of OpenMPI as a portable abstraction of the hierarchical topology of modern node architectures
- Showing the topology of the system: `hwloc-ls`

Example: Intel Core i7-1165G7



Valgrind Tools: Memcheck



- The default Valgrind Memcheck tool can be used to obtain the number of memory allocations:

```
==149464== Memcheck, a memory error detector
==149464== Copyright (C) 2002–2022, and GNU GPL'd, by Julian Seward et al.
==149464== Using Valgrind–3.22.0 and LibVEX; rerun with –h for copyright info
==149464== Command: ./a.out
==149464==
==149464==
==149464== HEAP SUMMARY:
==149464==     in use at exit: 0 bytes in 0 blocks
==149464==   total heap usage: 10 allocs, 10 frees, 245,760 bytes allocated
==149464==
==149464== All heap blocks were freed -- no leaks are possible
==149464==
==149464== For lists of detected and suppressed errors, rerun with: -s
==149464== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```c
#include <mcheck.h>
#include <stdlib.h>

double *a, *b;

int main() {
  for (int k = 1; k <= 5; k++) {
    const int m = sizeof(double);
    const int n = k * 1024;

    a = (double *) malloc(n * m);
    b = (double *) malloc(n * m);

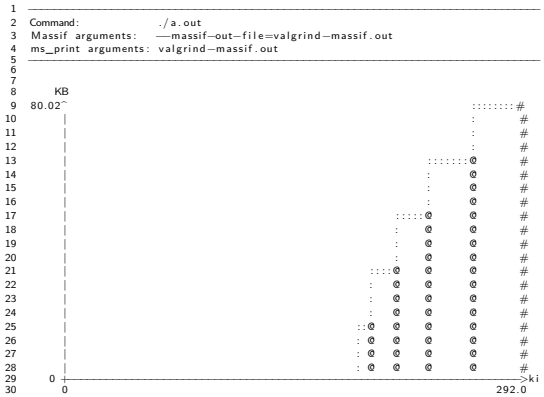
    for (int i = 0; i < n; i++)
      a[i] = b[i];

    free(b);
    free(a);
  }

  return 0;
}
```

Valgrind Tools: Massif

- Massif is a heap profiler
- Command line example: `valgrind --tool=massif ./a.out`
- Output parser: `ms_print`



Valgrind Tools: CacheGrind (1/2)



- Cachegrind is a tracing profiler including a cache simulator
- Command line example:

```
valgrind \
    --tool=cachegrind \
    --cache-sim=yes \
    --D1=65536,4,64 ./a.out
```

```
1  #define N 800000
2
3  double a[N], b[N];
4
5  int main()
6  {
7      for (int i = 0; i < N; i++)
8          a[i] = b[i];
9
10     return 0;
11 }
```

Valgrind Tools: CacheGrind (2/2)

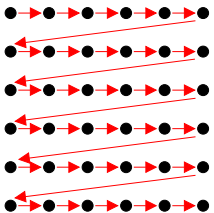
```
1  ==151340== Cachegrind, a high-precision tracing profiler
2  ==151340== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
3  ==151340== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
4  ==151340== Command: ./a.out
5  ==151340==
6  --151340-- warning: L3 cache found, using its data for the LL simulation.
7  ==151340==
8  ==151340== I refs:        4,178,640
9  ==151340== I1  misses:        1,131
10 ==151340== LLi misses:        1,123
11 ==151340== I1  miss rate:      0.03%
12 ==151340== LLi miss rate:      0.03%
13 ==151340==
14 ==151340== D refs:        1,651,120  (839,549 rd   + 811,571 wr)
15 ==151340== D1  misses:      201,495  (101,166 rd   + 100,329 wr)
16 ==151340== LLd misses:      201,421  (101,097 rd   + 100,324 wr)
17 ==151340== D1  miss rate:      12.2% (   12.1%     +    12.4%  )
18 ==151340== LLd miss rate:      12.2% (   12.0%     +    12.4%  )
19 ==151340==
20 ==151340== LL refs:          202,626  (102,297 rd   + 100,329 wr)
21 ==151340== LL misses:        202,544  (102,220 rd   + 100,324 wr)
22 ==151340== LL miss rate:         3.5% (    2.0%     +    12.4%  )
```

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

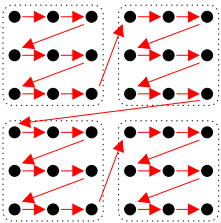
Cache Blocking



- **Cache blocking** is a strategy where data structures or memory access patterns are changed such that capacity cache misses are reduced
  - Optimization strategy sometimes also called **loop tiling** or **loop blocking**
- **Example**: Double-precision matrix-vector multiplication

$$y_i \leftarrow \sum_{j=0}^{N-1} A_{ij} x_j \quad (i = 0, \ldots, N-1)$$

  - Advantage: x needs to be loaded only once per block
  - Disadvantage: Non-linear read of A



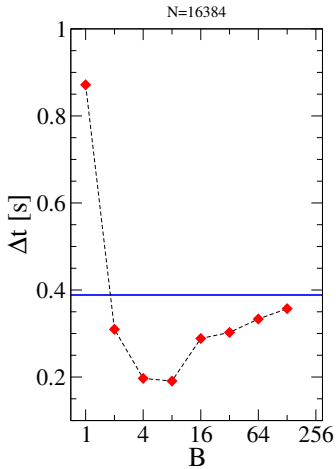
Cache-blocked Matrix-Vector Multiplication



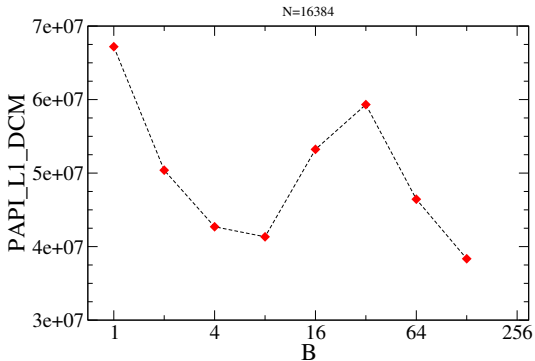
```
1  double x[N], y[N];
2  double A[N][N];
3
4  for (int ib = 0; ib < N/B; ib++) {
5    for (int i = 0; i < B; i++)
6      y[ib*B+i] = 0.0;
7
8    for (int jb = 0; jb < N/B; jb++)
9      for (int i = 0; i < B; i++)
10       for (int j = 0; j < B; j++) {
11         int ii = ib*B + i;
12         int jj = jb*B + j;
13         y[ii] += A[ii][jj] * x[jj];
14       }
15 }
```

Cache-blocked Matrix-Vector Multiplication

- Results obtained for $N = 16384$ on Intel Xeon E5-2623 v3 using a single thread
  - Horizontal line shows results for no cache blocking
- Observations
  - Large number of nested loops generate a significant overhead
  - Up to $2\times$ speed-up compared to no cache blocking
  - Best results are obtained for small block sizes with $B = 8$



Cache-blocked Matrix-Vector Multiplication



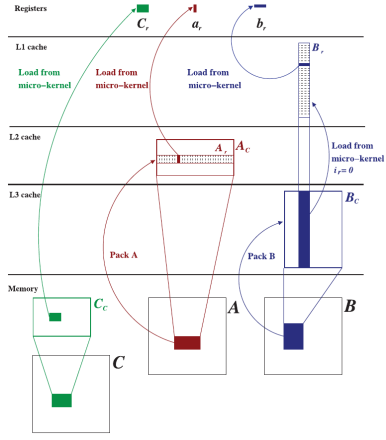
- For small B number of L1 data cache misses drop as expected
- The intermediate behaviour is possibly due to associativity

BLIS Design of gemm (BLAS-3)

- **gemm** = general matrix multiplication

$$C \leftarrow \alpha A B + \beta C$$

- BLIS = open source implementation of BLAS
- BLIS' gemm implementation is based on a systematic approach to multi-level blocking
  - Inner-most computations: hand-tuned micro-kernel
  - 5 outer loops
- Known analytic model for selecting implementation parameters based on hardware parameters
  - Cache: line size, associativity, size
  - SIMD instructions: throughput, width



[T.M. Low et al.]

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. **Dynamic Memory Allocation**
5. DTLB Optimization
6. Final

Dynamic Memory Allocation



- Possible issues related to dynamic memory allocation:
  - Increased number of page table updates
  - Lock contentions in case of multi-threading
  - Fragmentation
- Performance impact mitigation strategies
  - Minimize number of allocations/de-allocations
  - Use of optimised memory allocators, e.g. TCmalloc, jemalloc
  - Avoid small memory allocations

Alternative: TCmalloc



- TCMalloc is Google's customized implementation of `malloc()` provided by LIBC
- Benefits
  - Reduced lock contention
  - Improved thread scaling (depending on application)
- User-accessible controls
  - Logical page size: unit in which TCMalloc manages memory
  - Memory releasing
- Usage:
  `LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc.so ./myprog.x`

Alternative: jemalloc



- A replacement of `malloc()` that aims to reduce fragmentation and improve scalability
  - Became default in Facebook software and databases like Cassandra, MariaDB
- Benefits
  - Wall-clock-based purging of dirty pages (improved memory fairness)
  - Improved thread scaling (depending on application)
- User-accessible controls
  - Enabling background threads
  - Defining decay times for returning unused pages
  - Use of huge pages
- Usage:
  `LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so ./myprog.x`

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. Final

DTLB Optimizations



- Any algorithm that does random accesses into a large memory region will likely suffer from DTLB misses
- Example TLB sizes for Intel Golden Cover
  - 1st-level DTLB for load+store: 96+96 entries for 4k pages
  - 2nd-level DTLB: 1024 entries
- Performance impact mitigation strategies:
  - Optimize memory pattern (may not be possible)
  - Use of huge pages

Explicit Huge Pages



- Size of huge pages is ISA and/or processor dependent
  - Check `/proc/meminfo`
- Explicit huge pages can be reserved at system boot time or before an application starts
  - Example:

```
echo 20 \
    > /proc/sys/vm/nr_hugepages
```

- Memory allocation using `mmap` with flag `MAP_HUGETLB` set

Example output:

```
MemTotal:       32582900 kB
MemFree:         1834816 kB
MemAvailable:    8545752 kB
Buffers:          323880 kB
Cached:         10578596 kB
SwapCached:       829012 kB
...
HugePages_Total:      20
HugePages_Free:       20
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:           40960 kB
DirectMap4k:      511424 kB
DirectMap2M:    21225472 kB
DirectMap1G:    12582912 kB
```

Explicit Huge Pages Example



```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/mman.h>

int main()
{
    const int size = (1 << 26);      // 64 MiByte
    void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap() failed");
        return 1;
    }

    /* Some useful work */

    munmap(ptr, size);
    return 0;
}
```

Transparent Huge Pages



- Linux also offers **Transparent Huge Page Support (THP)**
- Check whether huge pages are enabled: `cat /sys/kernel/mm/transparent_hugepage/enabled`
  - `always`: Application requesting THP will stall on allocation failure
  - `madvise`: Like `always` but only for a particular address range
  - `never`
- `int madvise(void addr, size_t length, int advice)`
  - Linux system call used to give advice or directions to the kernel about an address range
  - Address range is defined by beginning (`addr`) and `length`
  - `advice` = `MADV_HUGEPAGE`: Enable THP for given address range

Transparent Huge Tables Example



```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/mman.h>

int main()
{
    const int size = (1 << 26);      // 64 MiByte
    void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap() failed");
        return 1;
    }

    madvise(ptr, size, MADV_HUGEPAGE);

    /* Some useful work */

    munmap(ptr, size);
    return 0;
}
```

Transparent Huge Tables: jemalloc



- Assume that huge pages are enabled with setting `madvise`
- Set environment variable `MALLOC_CONF="thp:always"` to inform jemalloc about the use of huge pages
  - Example command line:

```
LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so \
   MALLOC_CONF="thp:always" ./myprog.x
```

Which Huge Page Mechanism to Use?



- **Explicit Huge Pages**
  - Pro: EHPs are not affected by swapping
  - Con: Higher dependence on specific configuration of the target system
- **System-wide Transparent Huge Pages**
  - Pro: Good for quick tests
  - Con: May result in inefficient use of huge pages
- **Per-process Transparent Huge Pages** (`madvise`)
  - Pro: Flexible to use
  - Con: May be affected by swapping and costs of system calls

Content



1. Memory Architecture (2)
2. Memory Performance Tools
3. Cache Blocking
4. Dynamic Memory Allocation
5. DTLB Optimization
6. **Final**

Summary Lecture #10



- Performance relevant features of memory architectures
  - Caches
  - Non-uniform memory architectures
  - Virtual memory
- Tools for hardware discovery and memory-related monitoring and profiling
- Performance engineering topics
  - Cache blocking
  - Dynamic memory allocation
  - DTLB optimization and use of huge pages



[Wiki Media]

Questions?

These slides contain content and figures from

- Christoph Kessler, “Shared memory programming with OpenMP”, Lecture Slides, 2022
- A. Silberschatz, P.B. Galvin, G. Gagne, “Operating System Concepts, 10th Edition,” 2018